

CFRM 421/521

[Eunice Seung-hyun Han]

Homework 3

- **Due: Tuesday, May 13, 2025, 11:59 PM**
- Total marks: 38
- Late submissions are allowed, but a 20% penalty per day applies. Your last submission is considered for calculating the penalty.
- Use this Jupyter notebook as a template for your solutions. **Your solution must be submitted as both one Jupyter notebook and one PDF file on Gradescope.** There will be two modules on Gradescope, one for each file type. The notebook must be already run, that is, make sure that you have run all the code, save the notebook, and then when you reopen the notebook, checked that all output appears as expected. You are allowed to use code from the textbook, textbook website, or lecture notes.

1. Applying PCA [6 marks]

(a) [4 marks]

Train a SVM classifier with a Gaussian RBF kernel on the MNIST training data (the first 60,000 observations). Use the default arguments for the classifier. Take the first 10,000 observations as the training set given by `X_train_red` and `y_train_red` in the code below. Time your training using `%time` and report the accuracy of the trained model on the test set (the last 10,000 observations of MNIST). Now, use PCA on the original training set (60,000 observations) to find principal components with an explained variance ratio of 60%. Retrain the SVM classifier using these principal components instead of the original features. Time the training and accuracy on the test set. Comment on whether PCA helped or not. Do not shuffle the data and do not use a standard scaler.

```
In [ ]: import numpy as np

from sklearn.datasets import fetch_openml
mnist = fetch_openml('mnist_784', version=1, as_frame=False, cache=True)
mnist.target = mnist.target.astype(np.int8)
X_train = mnist["data"][:60000]
X_test = mnist["data"][60000:]
y_train = mnist["target"][:60000]
y_test = mnist["target"][60000:]
X_train_red = mnist["data"][:10000]
y_train_red = mnist["target"][:10000]
```

[Add your solution here]

```
In [ ]: import time
import numpy as np
from sklearn.datasets import fetch_openml
from sklearn.svm import SVC
from sklearn.decomposition import PCA
from sklearn.metrics import accuracy_score

# Baseline SVM on original pixels
clf = SVC(kernel='rbf') # RBF kernel, default C=1.0, gamma='scale'

print('Training time of a SVM classifier with a Gaussian RBF kernel:')
%time clf.fit(X_train_red, y_train_red) # single-run timing

y_pred = clf.predict(X_test)
orig_acc = accuracy_score(y_test, y_pred)
print(f"Original SVM classifier accuracy: {orig_acc}")
```

Training time of a SVM classifier with a Gaussian RBF kernel:
 CPU times: user 9.49 s, sys: 83.9 ms, total: 9.57 s
 Wall time: 9.68 s
 Original SVM classifier accuracy: 0.9594

```
In [ ]: # PCA → 60% explained variance, fit on all 60k
pca = PCA(n_components = 0.6)
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)
print("PCA components:", X_train_pca.shape[1])

# SVM on PCA-reduced data
clf_pca = SVC(kernel='rbf')
print('Training time of a SVM classifier on PCA-reduced data:')
%time clf_pca.fit(X_train_pca, y_train) # single-run timing

y_pred_pca = clf_pca.predict(X_test_pca)
pca_acc = accuracy_score(y_test, y_pred_pca)
print(f"SVM classifier on PCA-reduced data accuracy: {pca_acc}")
```

PCA components: 17

Training time of a SVM classifier on PCA-reduced data:

CPU times: user 10.9 s, sys: 124 ms, total: 11.1 s

Wall time: 11.3 s

SVM classifier on PCA-reduced data accuracy: 0.9723

The PCA did help the SVM classifier with a Gaussian RBF kernel to get higher accuracy score. When just using the SVM classifier with raw original data, the accuracy is 0.9594 and the SVM classifier on the PCA reduced data, got accuracy of 0.9723, which shows an improvement. However it also shows applying the PCA is more time consuming for training. The SVM classifier on the PCA model takes about x1.2 more CPU time than the original while fitting x6 larger training data (Original SVM training data - 10k & PCA reduced training data - 60k). If we assume both models are having the same training data set, then applying the PCA may take less time while giving higher accuracy score.

(b) [2 marks]

Repeat (a) using a random forest classifier instead of the SVM classifier and comment on the result. Use `random_state=42` in the classifier and use the default for the other arguments. You are allowed to use `n_jobs=-1`.

[Add your solution here]

```
In [ ]: from sklearn.ensemble import RandomForestClassifier

# Random Forest on original pixels
rf = RandomForestClassifier(random_state=42, n_jobs=-1)
%time rf.fit(X_train_red, y_train_red)

y_pred = rf.predict(X_test)
acc_raw = accuracy_score(y_test, y_pred)
print(f"Original Random Forest classifier accuracy: {acc_raw}")
```

CPU times: user 6.01 s, sys: 137 ms, total: 6.15 s

Wall time: 1.03 s

Original Random Forest classifier accuracy: 0.9504

```
In [ ]: # PCA → 60% explained variance
pca = PCA(0.60)
pca.fit(X_train)
X_train_pca = pca.transform(X_train)
X_test_pca = pca.transform(X_test)
print(f"PCA components retained: {X_train_pca.shape[1]}")

# Random Forest on PCA-reduced data
rf_pca = RandomForestClassifier(random_state=42, n_jobs=-1)
%time rf_pca.fit(X_train_pca, y_train)
```

```
y_pred_pca = rf_pca.predict(X_test_pca)
acc_pca = accuracy_score(y_test, y_pred_pca)
print(f"Random Forest classifier on PCA-reduced data accuracy: {acc_pca}")
```

PCA components retained: 17

CPU times: user 49.6 s, sys: 549 ms, total: 50.2 s

Wall time: 8.03 s

Random Forest classifier on PCA-reduced data accuracy: 0.9443

The PCA did not help the Random Forest classifier to gain higher accuracy as the accuracy of the original Random Forest classifier is higher. And also, the time takes much longer when training the PCA reduced data than using the original data.

So, not using the PCA reduced data in the Random Forest classifier would be more efficient.

2. Visualizing dimensionality reduction [8 marks]

(a) [3 marks]

Load the MNIST dataset and take only the first 5,000 observations as `X` and `y`. Use the nonlinear dimensionality reduction technique **t-SNE** to reduce this subset of the MNIST dataset down to 2 dimensions. Include the argument `random_state=42`. Measure how long it took to reduce the dimensionality, this will be used in (b) below. Plot a scatterplot of the outcome of t-SNE. Use 10 different colors to represent each image's target class and plot scaled-down versions of the digit images themselves. The visualization will be too cluttered if you draw the image for every observation, so you should either draw a random sample or plot an instance only if no other instance has already been plotted at a close distance. You should get a nice visualization of different classes and some of the digit images in them. See the textbook's Jupyter notebook for the code. Use the plot to comment on which classes are easily identified and which ones can be hard to distinguish.

Hint: See the solution of Chapter 8, Question 10 in this [Jupyter notebook](#) on the textbook website to see how to implement t-SNE and produce these plots using the `plot_digits()` function.

[Add your solution here]

```
In [ ]: X_sample, y_sample = X_train[:5000], y_train[:5000]
```

```
In [ ]: from sklearn.manifold import TSNE
```

```
tsne = TSNE(n_components=2, init="random", learning_rate="auto", random_state=42)
print('Time that took to reduce the dimensionality:')
%time X_reduced = tsne.fit_transform(X_sample)
```

Time that took to reduce the dimensionality:

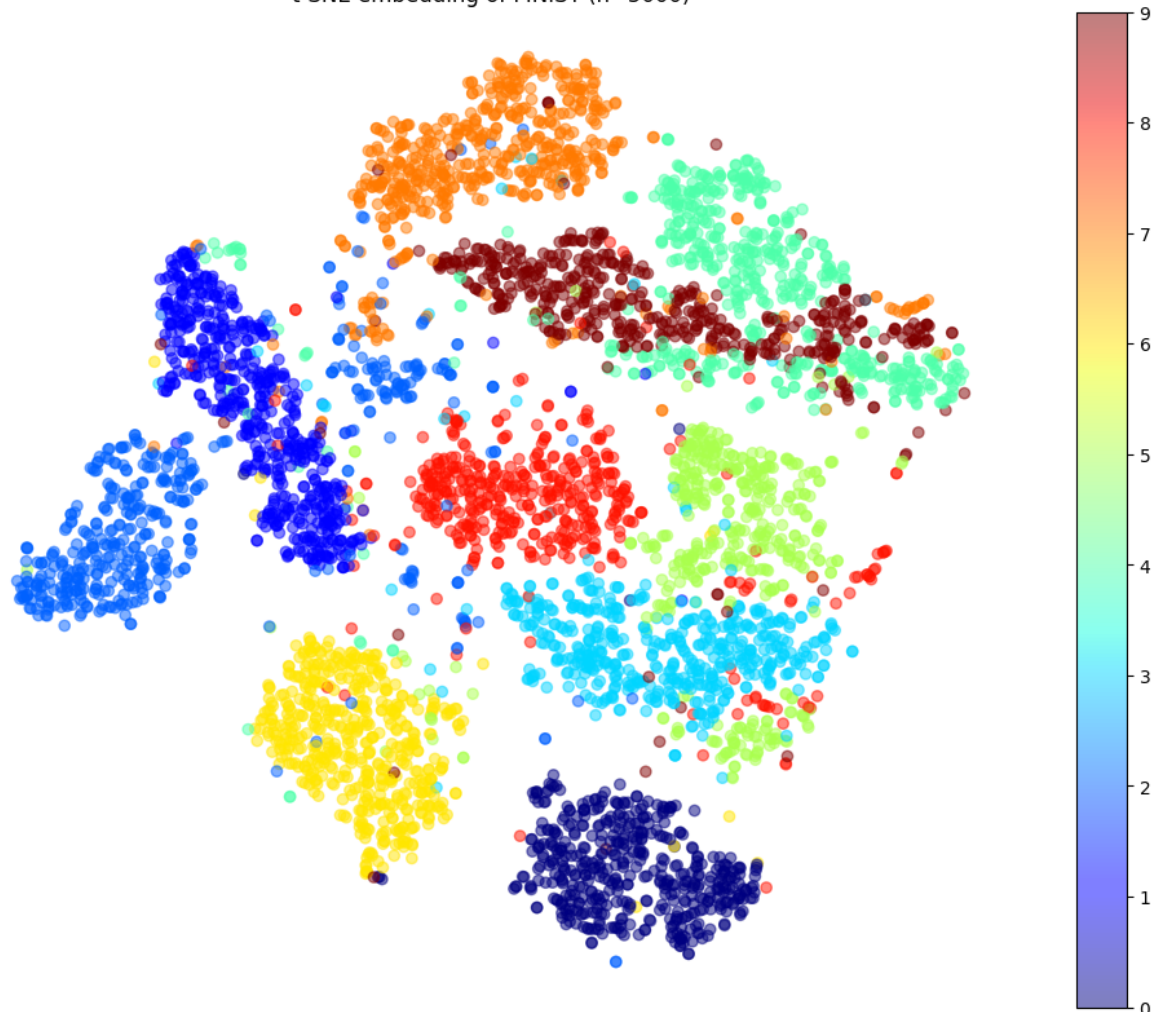
CPU times: user 1min 34s, sys: 11.9 s, total: 1min 46s

Wall time: 15.9 s

```
In [ ]: # plot a scatter plot
import matplotlib.pyplot as plt

plt.figure(figsize=(13, 10))
plt.scatter(X_reduced[:, 0], X_reduced[:, 1], c=y_sample.astype(np.int))
plt.axis('off')
plt.title("t-SNE embedding of MNIST (n=5000)")
plt.colorbar()
plt.show()
```

t-SNE embedding of MNIST (n=5000)



```

In [ ]: from sklearn.preprocessing import MinMaxScaler
from matplotlib.offsetbox import AnnotationBbox, OffsetImage

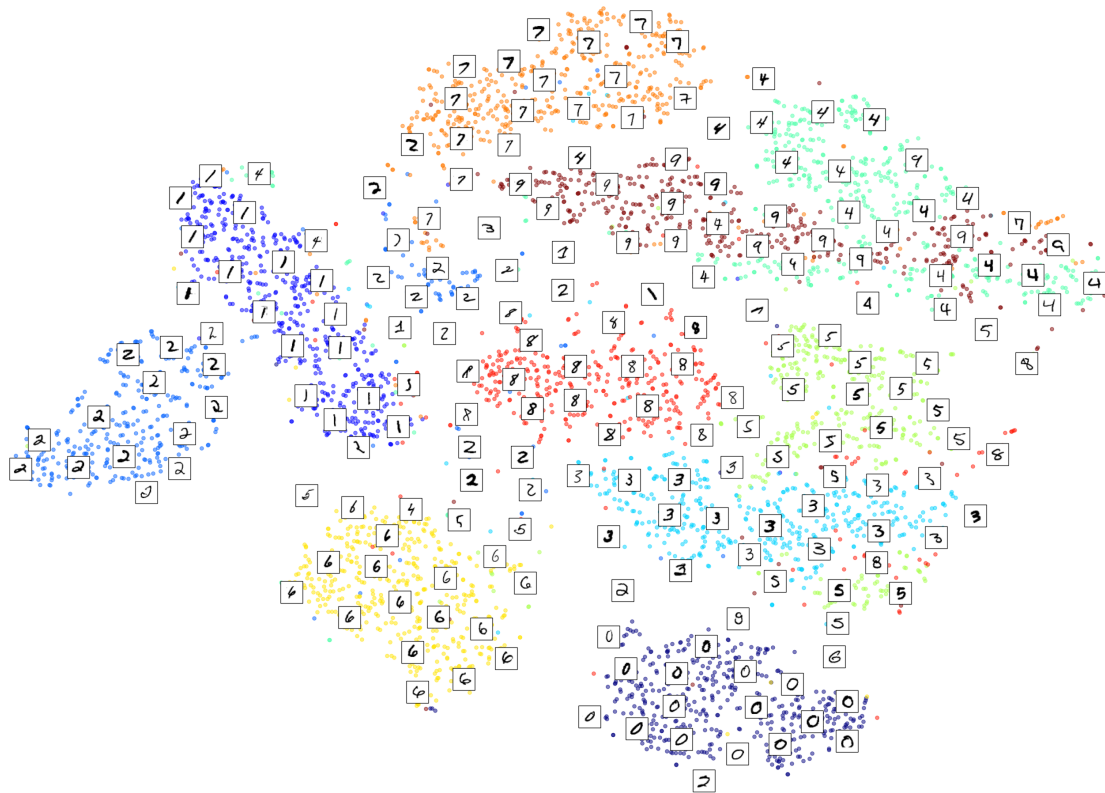
def plot_digits(X, y, min_distance=0.04, images=None, figsize=(13, 10)):
    X_normalized = MinMaxScaler().fit_transform(X)
    neighbors = np.array([[10., 10.]])
    plt.figure(figsize=figsize)
    cmap = plt.cm.jet
    digits = np.unique(y)
    for digit in digits:
        plt.scatter(X_normalized[y == digit, 0], X_normalized[y == digit, 1],
                    c=[cmap(float(digit) / 9)], alpha=0.5)
    plt.axis("off")
    ax = plt.gca()
    for index, image_coord in enumerate(X_normalized):
        closest_distance = np.linalg.norm(neighbors - image_coord, axis=1)
        if closest_distance > min_distance:
            neighbors = np.r_[neighbors, [image_coord]]
            if images is None:
                plt.text(image_coord[0], image_coord[1], str(int(y[index])),
                        color=cmap(float(y[index]) / 9),
                        fontdict={"weight": "bold", "size": 16})
            else:
                image = images[index].reshape(28, 28)
                imagebox = AnnotationBbox(OffsetImage(image, cmap="binary"),
                                         image_coord)
                ax.add_artist(imagebox)

```

```

In [ ]: plot_digits(X_reduced, y_sample, images=X_sample, figsize=(35, 25))

```



Using the plots, it shows which classes are easily identified and which ones can be hard to distinguish. The ones that is easily indentified is numbers 0, 6, and 7. The ones that can be hard to distinguish is 1, 2, 3, 4, 5, 8, and 9 because they are overlappings. Numbers 1, 2, 4, and 7 overlaps, 3 and 5 overlaps, and also 4 and 9 overlaps which makes it hard to distinguish the classes.

(b) [5 marks]

Try using other dimensionality reduction methods. Specifically, try:

- Regular PCA to reduce to 2 dimensions.
- LLE to reduce to 2 dimensions.
- First apply PCA with an explained variance ratio of 95%, then apply t-SNE to reduce to 2 dimensions.

For each algorithm, include the argument `random_state=42` . Then for each of the three methods above, report how long it took to reduce the dimension. Also, provide a 2D plot of the results. Which method runs faster? Which one results in a better visualization? Include t-SNE from (a) as part of your comparison.

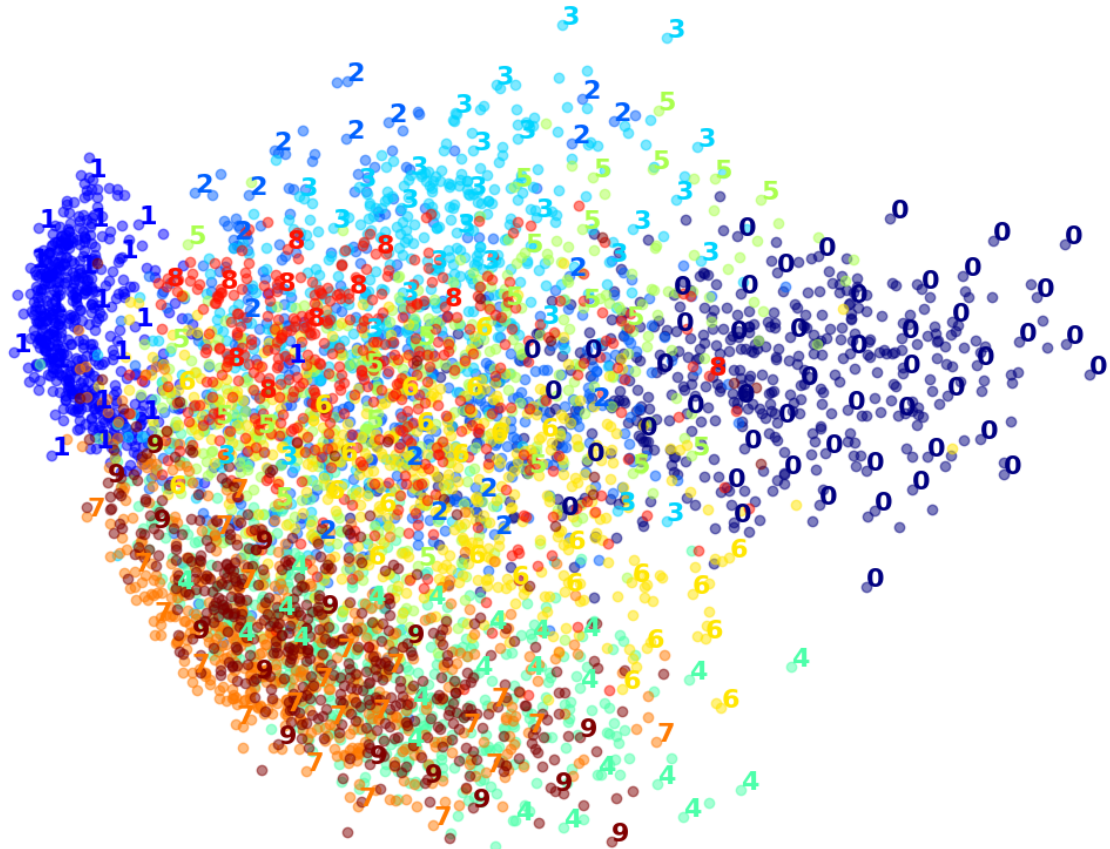
[Add your solution here]

```
In [ ]: # 1. Regular PCA to reduce to 2 dimensions

from sklearn.decomposition import PCA

pca2 = PCA(n_components=2, random_state=42)
print('Time that took to reduce the dimension:')
%time X_pca2 = pca2.fit_transform(X_sample)
plot_digits(X_pca2, y_sample)
```

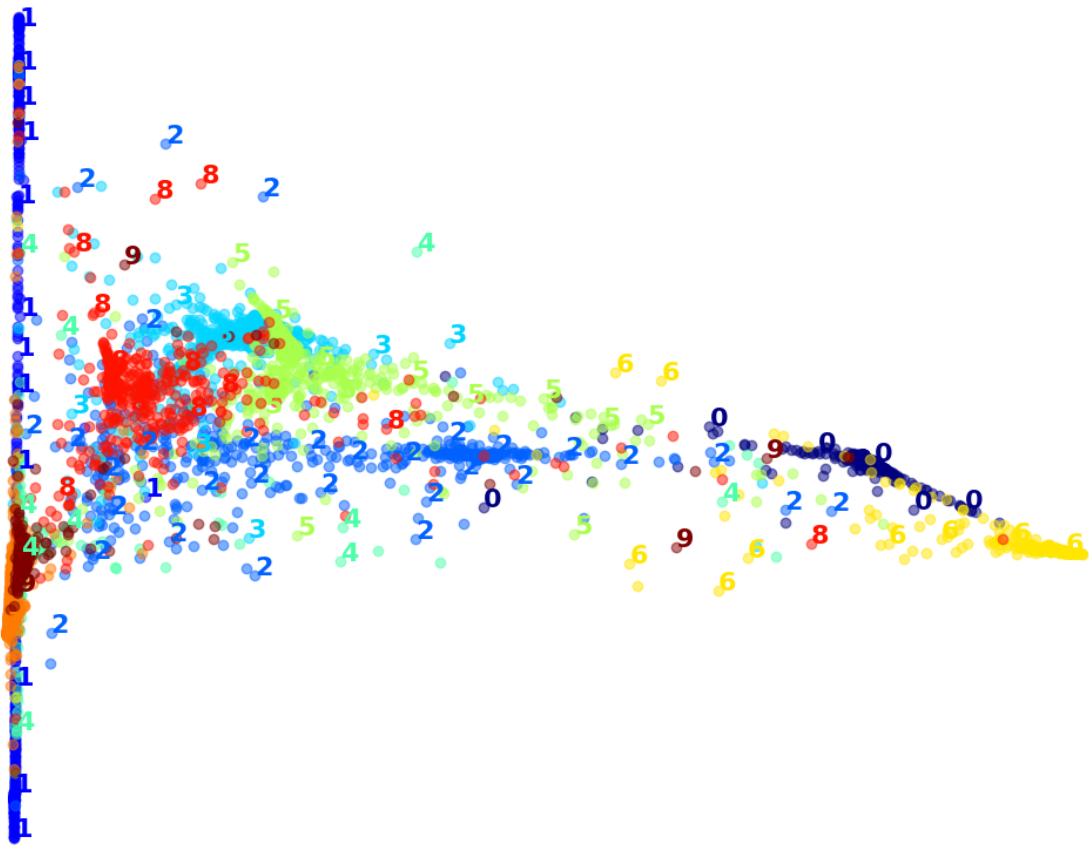
Time that took to reduce the dimension:
 CPU times: user 179 ms, sys: 23 ms, total: 202 ms
 Wall time: 95.7 ms



```
In [ ]: # 2. LLE to reduce to 2 dimensions

from sklearn.manifold import LocallyLinearEmbedding
lle = LocallyLinearEmbedding(n_components=2, random_state=42)
print('Time that took to reduce the dimension:')
%time X_lle = lle.fit_transform(X_sample)
plot_digits(X_lle, y_sample)
```

Time that took to reduce the dimension:
 CPU times: user 6.2 s, sys: 537 ms, total: 6.74 s
 Wall time: 2.17 s



In []: # 3. First apply PCA with an explained variance ratio of 95%, then app

```
pca95 = PCA(n_components=0.95, random_state=42)
print('Time that took to apply PCA with an explained variance ratio of
%time X_mid = pca95.fit_transform(X_sample)
tsne2 = TSNE(n_components=2, init="random", learning_rate="auto", rand

print('Time that took to reduce the dimension:')
%time X_pca_tsne = tsne2.fit_transform(X_mid)
plot_digits(X_pca_tsne, y_sample)
```

Time that took to apply PCA with an explained variance ratio of 95%:

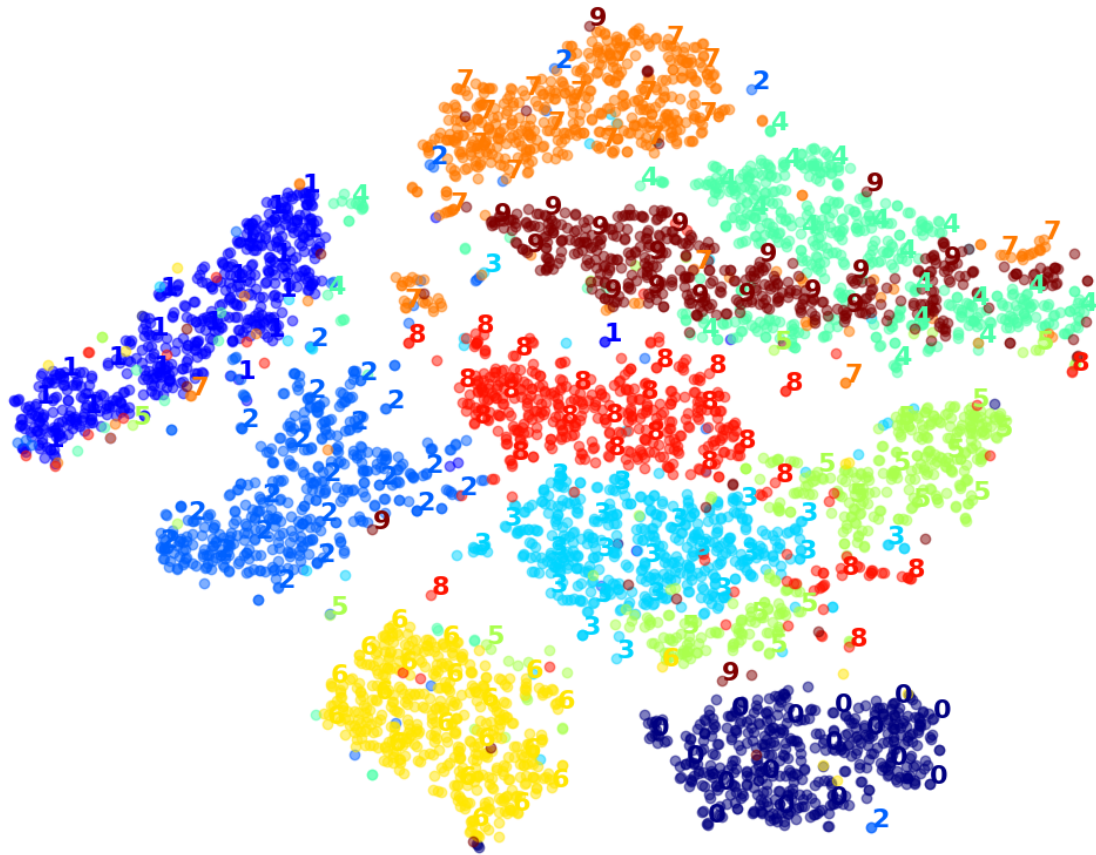
CPU times: user 2.23 s, sys: 190 ms, total: 2.42 s

Wall time: 725 ms

Time that took to reduce the dimension:

CPU times: user 1min 38s, sys: 11.7 s, total: 1min 50s

Wall time: 16.2 s



The method that runs fastest when reducing the dimensionality is, "regular PCA to reduce to 2 dimensions." And the method that results in a better visualization is, "first apply PCA with an explained variance ratio of 95%, then apply t-SNE to reduce to 2 dimensions." This method did have the slowest run time but provided clear and easy determined plots. We can see reducing the dimensions using t-SNE in both (a) and (b), showed the best visualization for identifying different classes.

3. k-Means clustering [11 marks]

Hint: It is helpful to read the solution of Chapter 9, Question 10 and 11 in this [Jupyter notebook](#).

(a) Chapter 9, Question 10 [6 marks]

The classic Olivetti faces dataset contains 400 grayscale 64×64 pixel images of faces. Each image is flattened to a 1D vector of size 4096. 40 different people were photographed (10 times each), and the task is to train a model that can predict which person is represented in each picture. Load the dataset and split it

into a training set and a validation set using the code below. Since the dataset is quite small, stratified sampling is used to ensure that there are the same number of images per person in each set. There is no need to create a test set.

```
In [ ]: from sklearn.datasets import fetch_olivetti_faces
from sklearn.model_selection import StratifiedShuffleSplit

data = fetch_olivetti_faces()
# Creating validation set
strat_split = StratifiedShuffleSplit(n_splits=1, test_size=120, random
train_idx, val_idx = next(strat_split.split(data.data, data.target))
X_valid, y_valid = data.data[val_idx], data.target[val_idx]
# Creating training set
X_train, y_train = data.data[train_idx], data.target[train_idx]
```

Note that the dataset is already scaled between 0 and 1, so there is no need to use a standard scaler. Next, use PCA on the features with an explained variance ratio of 99%. Then cluster the images based on the reduced features using k-Means, where the argument `random_state=42` is included. To determine the number of clusters $k \in \{5, 10, 15, \dots, 145\}$, (i) plot inertia, (ii) plot silhouette scores. Choose the number of clusters k' based on (ii). Then (iii) plot silhouette diagrams where the number of clusters is 40 and k' , and explain which number of clusters is better. Finally, suppose we use k' clusters, (iv) visualize the clusters by plotting the images in each cluster and comment on whether you see similar faces in each cluster.

[Add your solution here]

```
In [ ]: from sklearn.decomposition import PCA

pca = PCA(n_components = 0.99)
X_train_pca = pca.fit_transform(X_train)
X_valid_pca = pca.transform(X_valid)

pca.n_components_
```

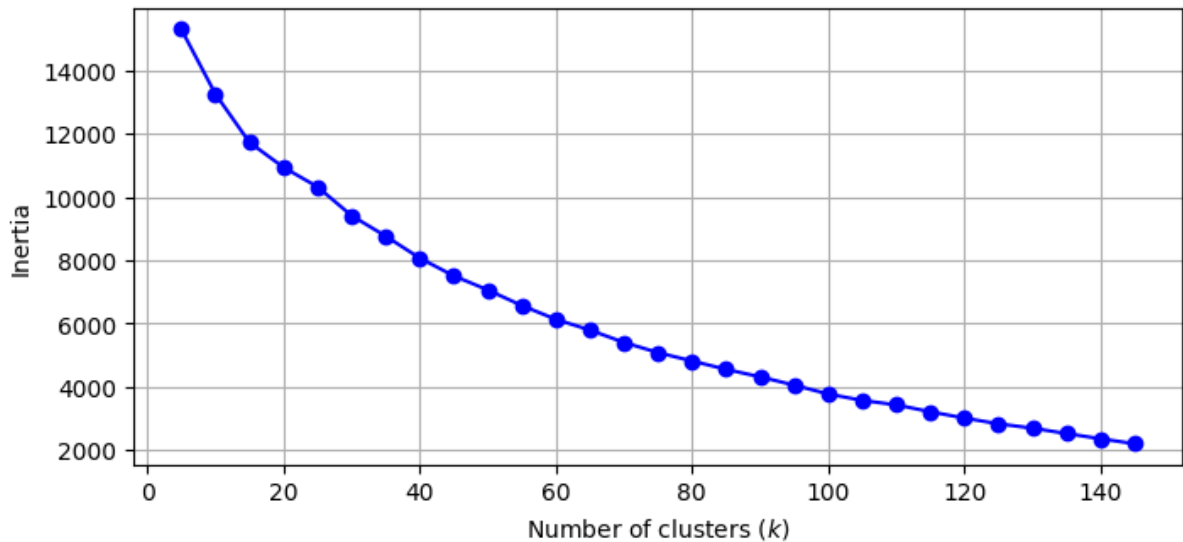
```
Out[ ]: np.int64(200)
```

```
In [ ]: from sklearn.cluster import KMeans

k_range = range(5, 150, 5)
kmeans_per_k = []
for k in k_range:
    #print(f"k={k}")
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X_train_pca)
    kmeans_per_k.append(kmeans)
```

```
In [ ]: # (i) plot inertia
inertias = [model.inertia_ for model in kmeans_per_k]
#best_inertia = inertias[best_index]

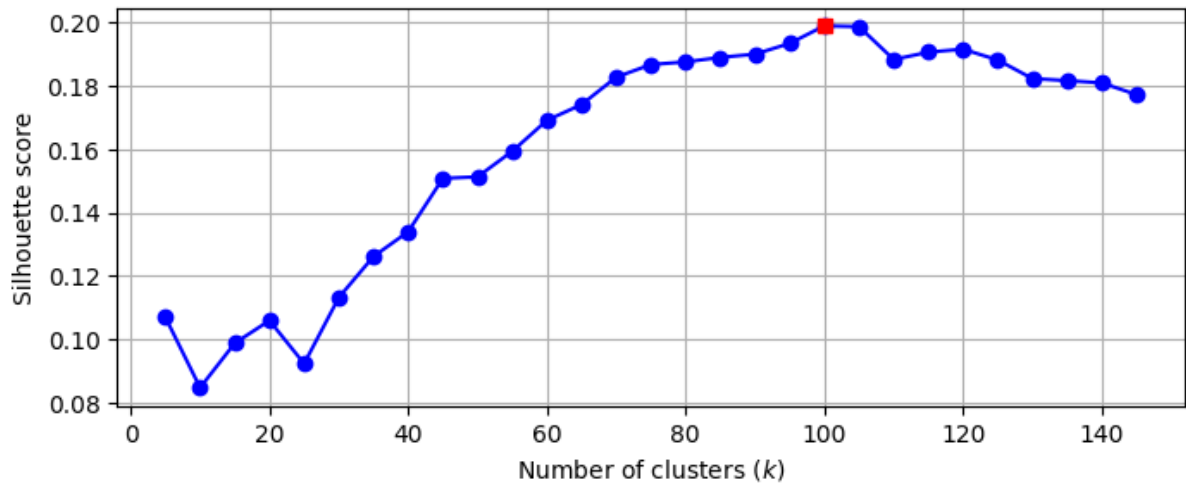
plt.figure(figsize=(8, 3.5))
plt.plot(k_range, inertias, "bo-")
plt.xlabel("Number of clusters ($k$)")
plt.ylabel("Inertia")
#plt.plot(best_k, best_inertia, "rs")
plt.grid()
plt.show()
```



```
In [ ]: # (ii) plot silhouette scores
from sklearn.metrics import silhouette_score

silhouette_scores = [silhouette_score(X_train_pca, model.labels_)
                      for model in kmeans_per_k]
best_index = np.argmax(silhouette_scores)
best_k = k_range[best_index]
best_score = silhouette_scores[best_index]

plt.figure(figsize=(8, 3))
plt.plot(k_range, silhouette_scores, "bo-")
plt.xlabel("Number of clusters ($k$)")
plt.ylabel("Silhouette score")
plt.plot(best_k, best_score, "rs")
plt.grid()
plt.show()
```



```
In [ ]: best_model = kmeans_per_k[best_index]
        print(best_model)
```

```
KMeans(n_clusters=100, random_state=42)
```

This tells $k' = 100$ as it provides the highest silhouette score.

```
In [ ]: # Choose the number of clusters k based on (ii) --> best_k (k' = 100)
        # (iii) plot silhouette diagrams where the number of clusters is 40 and
        #         the number of clusters is 100

import numpy as np
import matplotlib.pyplot as plt
import matplotlib.cm as cm
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_samples, silhouette_score

X_flat = X_train.reshape(len(X_train), -1)

k1 = 40
labels1 = KMeans(n_clusters=k1, random_state=42).fit_predict(X_flat)

k2 = len(np.unique(best_model.labels_))
labels2 = best_model.labels_

fig, axes = plt.subplots(1, 2, figsize=(10,10), sharey=False)

for ax, k, labels in zip(axes, (k1, k2), (labels1, labels2)):
    sil_avg = silhouette_score(X_flat, labels)
    sil_vals = silhouette_samples(X_flat, labels)

    ax.set_title(f"K = {k} (avg = {sil_avg:.3f})")
    ax.set_xlabel("Silhouette coefficient")
    ax.set_ylabel("Sample index (grouped by cluster)")

    ax.set_xlim(-0.1, 1)
```

```
ax.set_ylim(0, len(X_flat) + (k + 1) * 10)

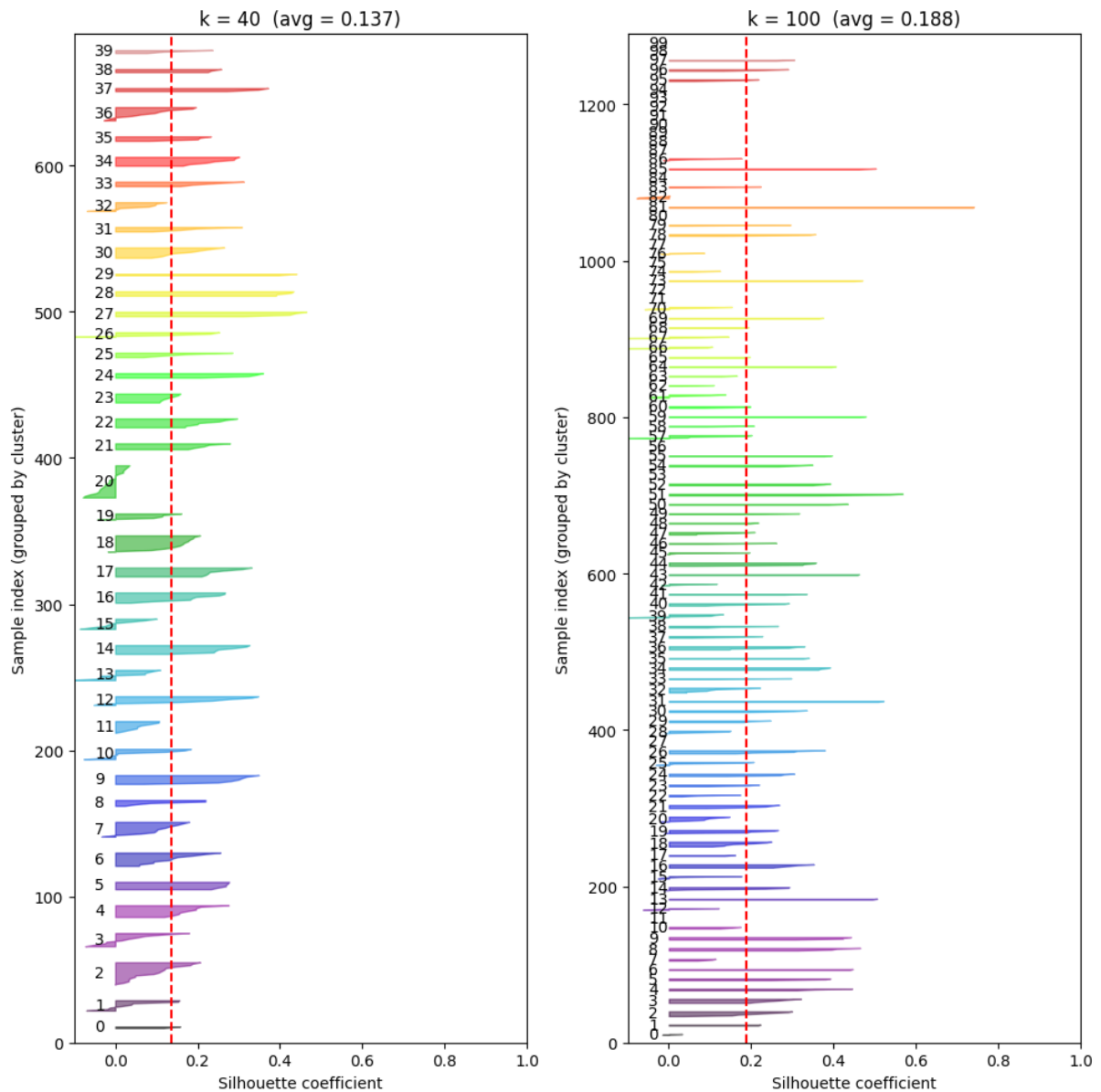
y_lower = 10
for i in range(k):
    ith_vals = np.sort(sil_vals[labels == i])
    size_i = ith_vals.shape[0]
    y_upper = y_lower + size_i

    color = cm.nipy_spectral(float(i) / k)
    ax.fill_betweenx(np.arange(y_lower, y_upper),
                     0, ith_vals,
                     facecolor=color, edgecolor=color, alpha=0.5)
    ax.text(-0.05, y_lower + 0.5 * size_i, str(i), va='center')

    y_lower = y_upper + 10

ax.axvline(x=sil_avg, color="red", linestyle="--")

plt.tight_layout()
plt.show()
```



When $k = 100$, it has a higher average silhouette score than $k = 40$. But looking on the graph, it seems like there are more narrow bars with negative silhouette coefficient when $k = 100$, which tells when $k = 40$, it has a more balanced clustering. Although the tight clusters in $k = 100$ outweigh the weaker ones and that's why the overall average silhouette coefficient is higher than $k = 40$ even though the clustering is less even.

In terms of the silhouette sense $k = 100$ would be better but since the goal is to recover 40 individuals, $k = 40$ would keep each person in one cluster and provide greater interpretability.

```
In [ ]: # (iv) visualize the clusters by plotting the images in each cluster

def plot_faces(faces, labels, n_cols=5):
    faces = faces.reshape(-1, 64, 64)
    n_rows = (len(faces) - 1) // n_cols + 1
```

```
plt.figure(figsize=(n_cols, n_rows * 1.1))
for index, (face, label) in enumerate(zip(faces, labels)):
    plt.subplot(n_rows, n_cols, index + 1)
    plt.imshow(face, cmap="gray")
    plt.axis("off")
    plt.title(label)
plt.show()

for cluster_id in np.unique(best_model.labels_):
    print("Cluster", cluster_id)
    in_cluster = best_model.labels_==cluster_id
    faces = X_train[in_cluster]
    labels = y_train[in_cluster]
    plot_faces(faces, labels)
```

Cluster 0

0



15



Cluster 1

22



22



Cluster 2

38



38



38



38



38



38



38



Cluster 3



Cluster 4



Cluster 5



Cluster 6



Cluster 7



Cluster 8



Cluster 9

36



36



36



36



Cluster 10

29



29



29



Cluster 11

4



Cluster 12

1



1



1



Cluster 13

9



9



Cluster 14

14



14



14



12



14



Cluster 15

4



4



4



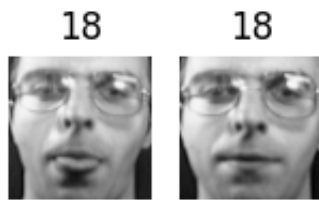
2



Cluster 16



Cluster 17



Cluster 18



Cluster 19



Cluster 20



Cluster 21

10



10



10



10



10



Cluster 22

31



31



31



Cluster 23

0



39



39



Cluster 24

16



16



16



16



Cluster 25

8



11



8



8



8



Cluster 26

26



26



26



26



26



Cluster 27

7



Cluster 28

6



6



6



6



Cluster 29

11



11



11



Cluster 30

36



36



36



Cluster 31

33



33



Cluster 32

24



24



24



24



24



24



24



Cluster 33

27



27



Cluster 34

30



30



30



30



Cluster 35

34



34



Cluster 36

5



5



5



5



5



Cluster 37

27



27



27



Cluster 38

21



21



21



Cluster 39

34



20



20



20



20



Cluster 40

1



1



1



1



Cluster 41

7

7



Cluster 42

37

37

25



Cluster 43

6

6



Cluster 44

37

37

37

37

37



Cluster 45

9

9

9



Cluster 46

19

19



Cluster 47

29



29



29



29



Cluster 48

19



19



Cluster 49

19



19



Cluster 50

23



23



Cluster 51

18



18



18



Cluster 52

20



20



20



Cluster 53

0



Cluster 54

14



14



14



Cluster 55

39



39



Cluster 56

35



Cluster 57

28



28



28



28



28



Cluster 58

28



28



Cluster 59

15



15



Cluster 60

12



12



12



Cluster 61

25



25



25



25



16



Cluster 62

9



9



Cluster 63

3



3



Cluster 64

31



31



Cluster 65

2



25



Cluster 66

15



0



15



Cluster 67

32



32



32



Cluster 68

35



35



Cluster 69

16



16



Cluster 70

4



4



4



8



Cluster 71

2



Cluster 72

7



Cluster 73

2



2



Cluster 74

11



11



Cluster 75

23



Cluster 76

31

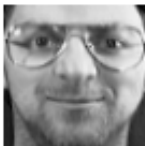


31



Cluster 77

6



Cluster 78

34



34



34

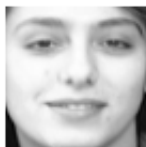


Cluster 79

7



7



Cluster 80

34



Cluster 81

27



27



Cluster 82

12



22



17



8



Cluster 83

26



26



Cluster 84

21



Cluster 85

18



18



Cluster 86

39



39



2



Cluster 87

12



Cluster 88

0



Cluster 89

19



Cluster 90

2



Cluster 91

7



Cluster 92

0



Cluster 93

35



Cluster 94

11



Cluster 95

21



21



21



Cluster 96

3



3



3



Cluster 97

10



10



Cluster 98

15



Cluster 99

0



In each cluster I see similar faces of individuals. Some clusters have different individuals mixed but mostly, each cluster has similar faces.

(b) [2 marks]

Continue to use the reduced features based on PCA in (a). Train a random forest classifier `RandomForestClassifier(n_estimators=150, random_state=42)` and a histogram-based gradient boosting classifier using `HistGradientBoostingClassifier(max_iter=200, random_state=42, early_stopping=True, validation_fraction=0.2)` to predict which person is represented in each picture, and evaluate it on the validation set. For the gradient boosting classifier state the optimal number of iterations from early stopping. State which of the two models is better.

[Add your solution here]

```
In [ ]: from sklearn.ensemble import RandomForestClassifier, HistGradientBoost
        clf = RandomForestClassifier(n_estimators=150, random_state=42)
        clf.fit(X_train_pca, y_train)
        clf_score = clf.score(X_valid_pca, y_valid)

        print(f"RandomForestClassifier validation accuracy: {clf_score}")
```

RandomForestClassifier validation accuracy: 0.9333333333333333

```
In [ ]: hgb = HistGradientBoostingClassifier(max_iter=200, random_state=42, ea
        hgb.fit(X_train_pca, y_train)

        # optimal number of iterations chosen by early stopping:
        opt_iter = hgb.n_iter_
        print(f"Optimal number of iterations from early stopping: {opt_iter}")
        hgb_score = hgb.score(X_valid_pca, y_valid)

        print(f"HistGradientBoostingClassifier validation accuracy: {hgb_score}")
```

Optimal number of iterations from early stopping: 85

HistGradientBoostingClassifier validation accuracy: 0.7666666666666667

HistGradientBoostingClassifier validation accuracy: 0.7666666666666667

Comparing the two models, 'Random Forest Classifier' and 'Histogram-based

Gradient Boosting Classifier,' the 'Random Forest Classifier' is better as it provides higher validation accuracy score.

(c) Chapter 9, Question 11 [4 marks]

Continuing on from (b), regardless of which model is better, use the random forest classifier. Next, use k-Means as a dimensionality reduction tool, and train a classifier. Search for the number of clusters $k \in \{5, 10, 15, \dots, 145\}$ that allows the classifier to get the best performance on the validation set. What performance can you reach on the validation set? What if you append the features from the reduced set to the original features and again search for the best number of clusters?

[Add your solution here]

```
In [ ]: from sklearn.pipeline import make_pipeline

total = []
best_score_kmeans = 0
best_k_kmeans = None

for n_clusters in k_range:
    pipeline = make_pipeline(KMeans(n_clusters = n_clusters, random_st
    pipeline.fit(X_train_pca, y_train)
    score = pipeline.score(X_valid_pca, y_valid)
    total.append(score)
    print(n_clusters, score)
    if score > best_score_kmeans:
        best_score_kmeans = score
        best_k_kmeans = n_clusters
```

```

5 0.3916666666666666
10 0.6083333333333333
15 0.6416666666666667
20 0.6833333333333333
25 0.7166666666666667
30 0.725
35 0.7416666666666667
40 0.7416666666666667
45 0.7416666666666667
50 0.7583333333333333
55 0.7583333333333333
60 0.7666666666666667
65 0.7416666666666667
70 0.775
75 0.8083333333333333
80 0.8083333333333333
85 0.7916666666666666
90 0.8
95 0.8
100 0.7666666666666667
105 0.7833333333333333
110 0.8166666666666667
115 0.775
120 0.7666666666666667
125 0.8166666666666667
130 0.8166666666666667
135 0.8083333333333333
140 0.7833333333333333
145 0.775

```

```

In [ ]: total = np.array([total])
        average = np.average(total)
        print("The average accuracy without extension: ", average)
        print(f"\nBest KMeans-only accuracy: {best_score_kmeans:.17f} at k = {

```

The average accuracy without extension: 0.7485632183908046

Best KMeans-only accuracy: 0.8166666666666665 at k = 110

```

In [ ]: total = []
        best_score_extended = 0
        best_k_extended = None

        for n_clusters in k_range:
            model = KMeans(n_clusters=n_clusters, random_state=42)
            model.fit(X_train_pca)

            X_train_reduced = model.fit_transform(X_train_pca)
            X_valid_reduced = model.transform(X_valid_pca)

            X_train_extended = np.c_[X_train_pca, X_train_reduced]
            X_valid_extended = np.c_[X_valid_pca, X_valid_reduced]

```

```

clf = RandomForestClassifier(n_estimators=150, random_state=42)

clf.fit(X_train_extended, y_train)
score = clf.score(X_valid_extended, y_valid)
total.append(score)
print(n_clusters, score)

if score > best_score_extended:
    best_score_extended = score
    best_k_extended = n_clusters

```

```

5 0.9083333333333333
10 0.9416666666666667
15 0.8916666666666667
20 0.9
25 0.8833333333333333
30 0.875
35 0.9083333333333333
40 0.9
45 0.8833333333333333
50 0.8833333333333333
55 0.9
60 0.85
65 0.8666666666666667
70 0.9166666666666666
75 0.85
80 0.8833333333333333
85 0.8833333333333333
90 0.8583333333333333
95 0.8666666666666667
100 0.8833333333333333
105 0.8916666666666667
110 0.9
115 0.85
120 0.875
125 0.8916666666666667
130 0.8833333333333333
135 0.8833333333333333
140 0.875
145 0.8583333333333333

```

```

In [ ]: total = np.array(total)
        average = np.average(total)
        print("The average accuracy with extension: ", average)
        print(f"\nBest Extended accuracy: {best_score_extended:.17f} at k = {b

```

The average accuracy with extension: 0.8841954022988505

Best Extended accuracy: 0.9416666666666665 at k = 10

The best performance we reached on the validation set when using k-Means as a dimensionality reduction tool, and train a classifier is accuracy:

0.8166666666666665 at cluster, $k = 110$. And the average accuracy is 0.7485632183908046. When I appended the features from the reduced set to the original features and again search for the best number of clusters, the best accuracy is 0.9416666666666665 at cluster, $k = 10$. And the average accuracy is 0.8841954022988505.

This shows, appending the features from the reduced set to the original features retains a more higher accuracy.

4. Finding regimes in time series [12 marks]

(a) [6 marks]

Obtain the daily values of the CPI and unemployment rate from FRED up to 2023-01-01 and then convert the CPI into the yearly inflation rate `inf_data` using the following code. Note that you may have to install the package `pandas_datareader`. Alternatively, you can download the data as a csv file from [Canvas](#).

```
In [ ]: #from pandas_datareader.data import DataReader
import pandas_datareader as pdr
import pandas as pd
from datetime import datetime

unemp_data = pdr.get_data_fred('UNRATE', datetime(1945,1,1),datetime(2
cpi_data = pdr.get_data_fred('CPIAUCSL', datetime(1945,1,1),datetime(2
inf_data = (cpi_data/cpi_data.shift(12)-1).iloc[12:]
data = pd.concat([inf_data, unemp_data],axis=1)
```

Split the data into a training set (before Jan 2005, using `data[:"2005-1-1"]` for instance) and a test set (Jan 2005 and after). Now, ignore the time aspect of your training set by treating it as a 2-dimensional dataset. Use k-means to divide your training set into a number of clusters called **regimes**. To find the number of regimes (clusters) $k \in \{2, 3, \dots, 10\}$: (i) plot inertia, (ii) plot silhouette scores, (iii) plot silhouette diagrams for $k \in \{2, 3, \dots, 10\}$. How many regimes do you choose? Explain your answer. Recall that you should use a standard scaler before applying k-means.

[Add your solution here]

```
In [ ]: from sklearn.preprocessing import StandardScaler
```

```

# (i) plot inertia

train = data[:'2005-01-01']
test = data['2005-01-01':]

X_train = train.values

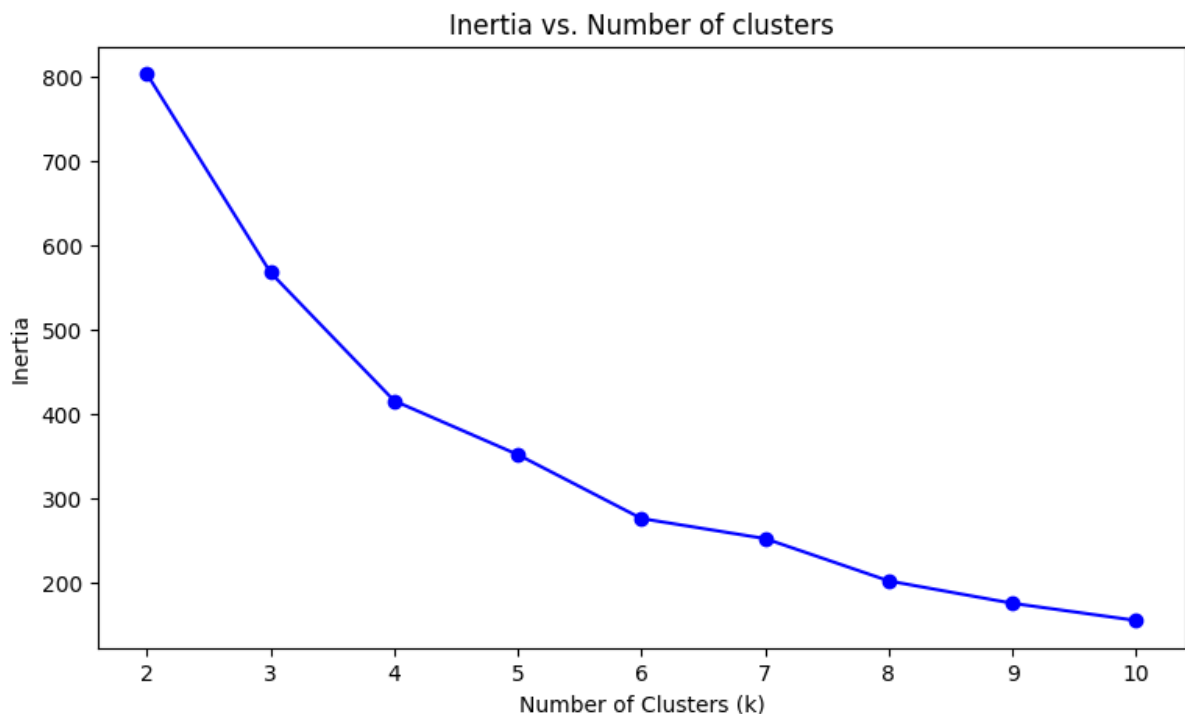
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_train)

k_range = range(2, 11)
inertias = []
sil_scores = []

kmeans_per_k = [KMeans(n_clusters=k, random_state=42).fit(X_scaled) for k in k_range]
inertias = [model.inertia_ for model in kmeans_per_k]

plt.figure(figsize=(9,5))
plt.plot(k_range, inertias, "bo-")
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia')
plt.title('Inertia vs. Number of clusters')
plt.show()

```



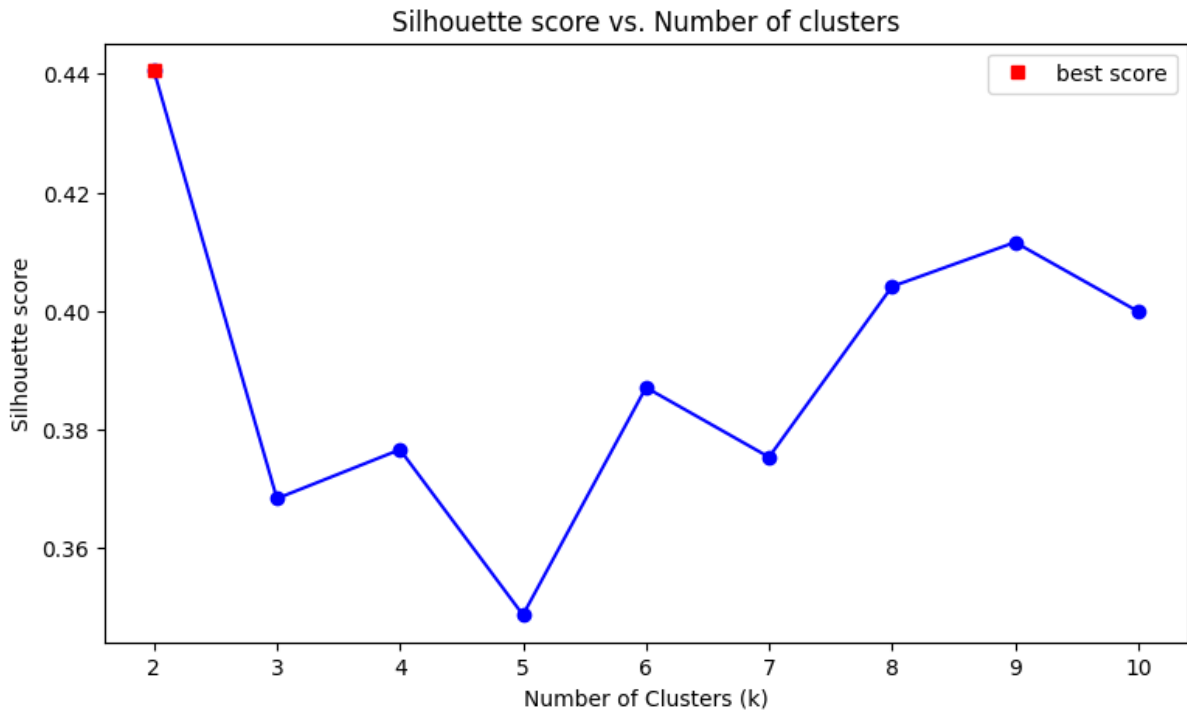
```

In [ ]: # (ii) plot silhouette scores

silhouette_scores = [silhouette_score(X_scaled, model.labels_) for model in kmeans_per_k]
best_index = np.argmax(silhouette_scores)
best_k = k_range[best_index]
best_score = silhouette_scores[best_index]

```

```
plt.figure(figsize=(9,5))
plt.plot(k_range, silhouette_scores, "bo-")
plt.plot(best_k, best_score, 'rs', label='best score')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Silhouette score')
plt.title('Silhouette score vs. Number of clusters')
plt.legend()
plt.show()
```



```
In [ ]: # (iii) plot silhouette diagrams

from matplotlib import cm

cmap = cm.get_cmap('coolwarm')

fig, axes = plt.subplots(3, 3, figsize=(13, 12), sharex=False, sharey=False)
axes = axes.flatten()

for idx, (ax, k, model) in enumerate(zip(axes, k_range, kmeans_per_k)):
    labels = model.labels_
    sil_vals = silhouette_samples(X_scaled, labels)

    y_lower = 10
    for i in range(k):
        vals = np.sort(sil_vals[labels == i])
        size_i = len(vals)
        y_upper = y_lower + size_i

        color = cmap(i / (k - 1))
        ax.fill_betweenx(
            np.arange(y_lower, y_upper),
```

```

        0, vals,
        facecolor=color, edgecolor=color, alpha=0.7
    )
    y_lower = y_upper + 10

    ax.axvline(x=silhouette_scores[idx], color='red', linestyle='--')

    ax.set_xlim([-0.1, 1])
    ax.set_ylim([0, len(X_scaled) + (k + 1) * 10])
    ax.set_title(f"$k={k}$", fontsize=14)

    if idx % 3 == 0:
        ax.set_ylabel("Cluster")
    else:
        ax.set_yticks([])

    if idx // 3 == 2:
        ax.set_xlabel("Silhouette coefficient")
    else:
        ax.set_xticklabels([])

    for j in range(len(k_range), 9):
        fig.delaxes(axes[j])

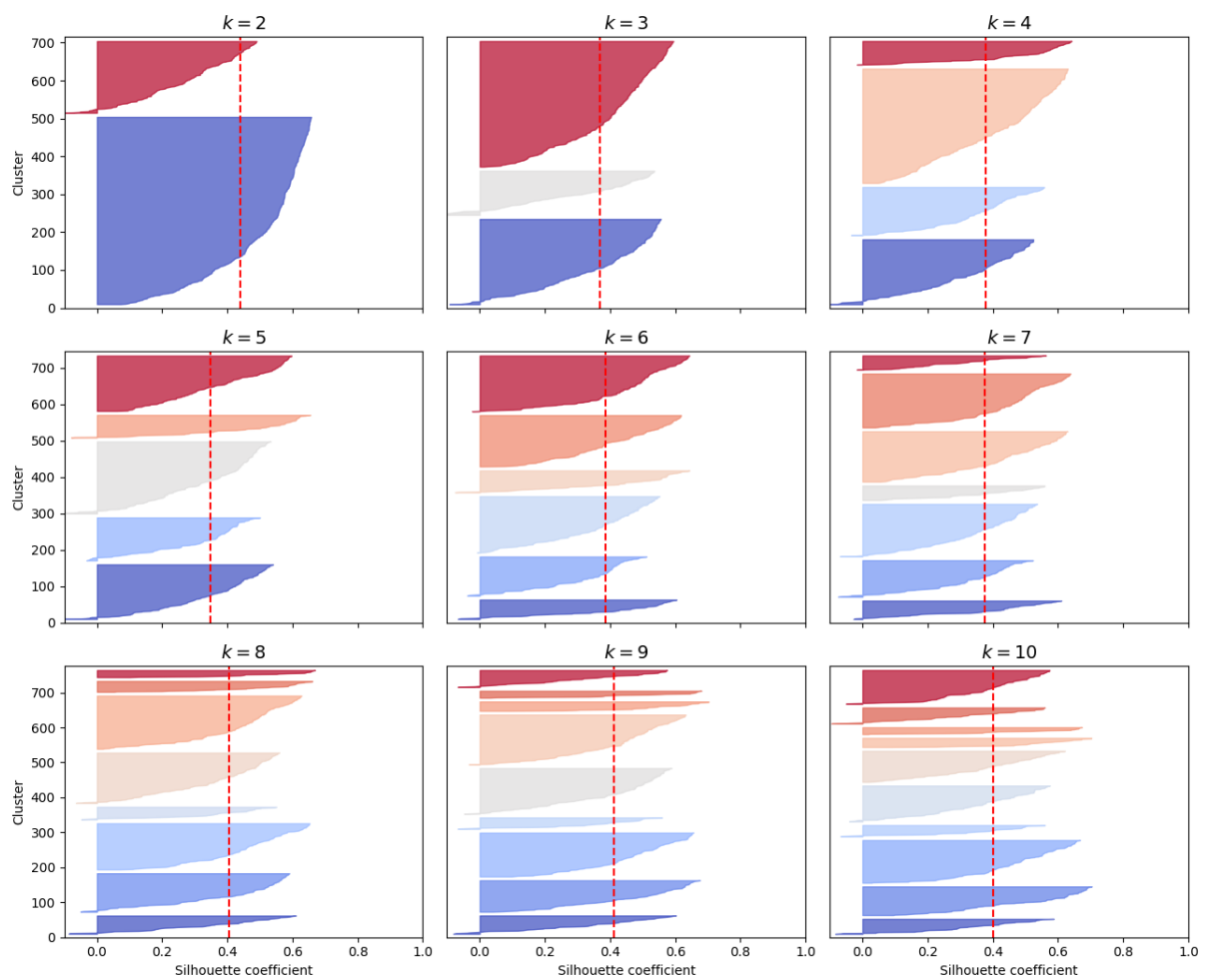
    fig.suptitle(r"Silhouette Diagrams for $k=2,\dots,10$", fontsize=18, y
    plt.tight_layout(rect=[0, 0.03, 1, 0.95])
    plt.show()

```

```

/var/folders/lc/16gprrls4_zff4bwgxvw2dnc0000gn/T/ipykernel_9937/3749636
170.py:5: MatplotlibDeprecationWarning: The get_cmap function was depre
cated in Matplotlib 3.7 and will be removed in 3.11. Use ``matplotlib.c
olormaps[name]`` or ``matplotlib.colormaps.get_cmap()`` or ``pyplot.get
_cmap()`` instead.
    cmap = cm.get_cmap('coolwarm')

```

Silhouette Diagrams for $k = 2, \dots, 10$ 

I will choose $k=2$ regimes because in the 'Silhouette score vs. Number of clusters' plot it shows it has the highest Silhouette score when $k=2$. Also in the Silhouette

diagram above, $k=2$ has almost all samples to the right of zero and the red dashed line (average coefficient) sitting highest. Other regimes shows the bars being in the negative side and also having lower red dash lines which represent the overall mean silhouette score is smaller.

(b) [3 marks]

Report the centroid of the regimes in (a). For the training set, plot the inflation rate (x-axis) vs unemployment rate (y-axis) after these features have been standardized, and show the regimes and centroids in the plot. For the test set, plot time series of the inflation rate and unemployment rate (the original series before standardization), and show the regimes on the plot. The regimes can be shown using time series (line graphs) of the centroid for each of two series for instance, and the following code can be used, where `best_mod_pred` is the prediction on the test set from the best model.

```
In [ ]: # regimes_train = pd.DataFrame(data_test.iloc[:,1]).assign(
#       regime = clusters[best_mod_pred][:,1].flatten()
# )

# plt.figure(figsize=(12,6))
# plt.plot(regimes_train["UNRATE"], label="vol", alpha=.3)
# plt.plot(regimes_train["regime"], '-k', label="regime")
```

[Add your solution here]

```
In [ ]: # Report the centroid of the regimes in (a)

best_model = kmeans_per_k[0] # when regimes = 2
centroids_std = best_model.cluster_centers_
print("Best model:", best_model)
print("Centroid when regimes = 2:")
print(centroids_std)
print(f"Inertia: {best_model.inertia_:.15f}")
```

```
Best model: KMeans(n_clusters=2, random_state=42)
Centroid when regimes = 2:
[[-0.36733388 -0.42687218]
 [ 0.95700143  1.11211437]]
Inertia: 804.005231132915242
```

```
In [ ]: # For the training set, plot the inflation rate (x-axis) vs unemployme
# after these features have been standardized, and show the regimes an

labels_train = best_model.labels_

def plot_data(train_scaled):
```

```

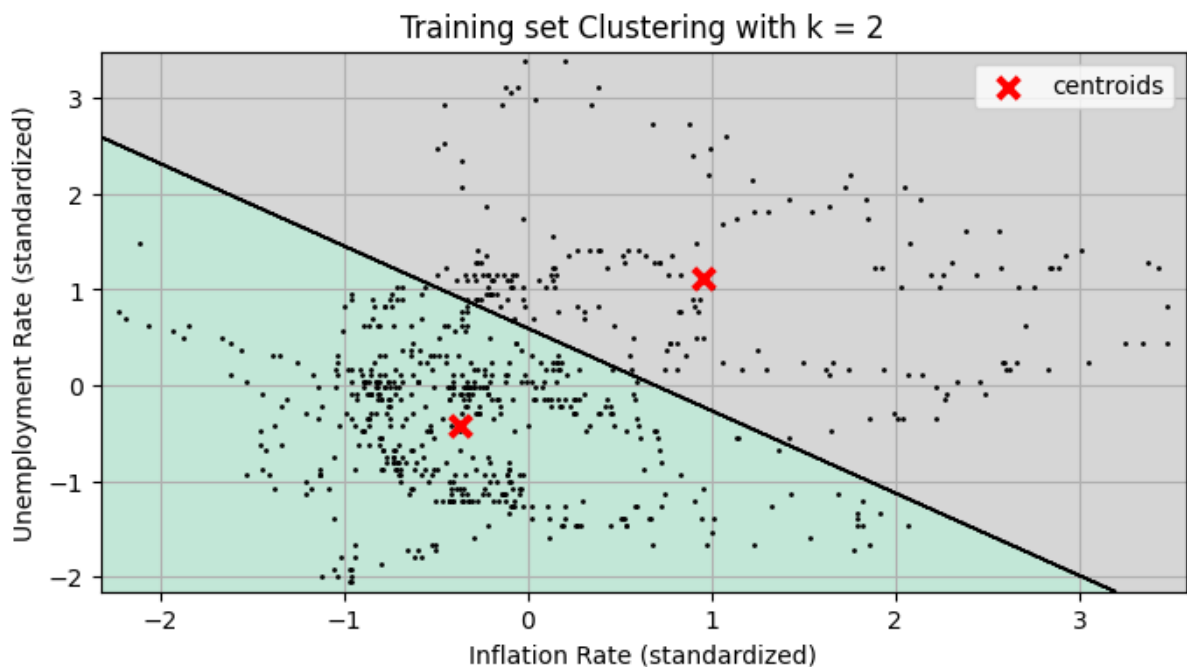
plt.plot(train_scaled[:, 0], train_scaled[:, 1], 'k.', markersize=
def plot_centroids(centroids, weights=None, circle_color='blue', cross
    if weights is not None:
        centroids = centroids[weights > weights.max() / 10]
    plt.scatter(centroids[:, 0], centroids[:, 1],
                marker='x', s=70, linewidths=3,
                color=cross_color, zorder=11, alpha=1, label='centroid

def plot_decision_boundaries(clusterer, train_scaled, resolution=1000)
    mins = train_scaled.min(axis=0) - 0.1
    maxs = train_scaled.max(axis=0) + 0.1
    xx, yy = np.meshgrid(np.linspace(mins[0], maxs[0], resolution), np
    Z = clusterer.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    plt.contourf(xx, yy, Z, cmap="Pastel2", alpha=0.8)
    plt.contour(xx, yy, Z, linewidths=1, colors='k')
    plot_data(train_scaled)
    plot_centroids(clusterer.cluster_centers_)
    plt.xlabel("Inflation Rate (standardized)")
    plt.ylabel("Unemployment Rate (standardized)")
    plt.title(f"Training set Clustering with k = {clusterer.n_clusters}
    plt.grid(True)

plt.figure(figsize=(8, 4))
plot_decision_boundaries(best_model, X_scaled)
plt.legend()
plt.show()

```



```

In [ ]: # For the test set, plot time series of the inflation rate and unemplo
        # (the original series before standardization), and show the regimes o

```

```

X_test = test.values
X_test_scale = scaler.transform(X_test)

best_model.fit(X_scaled)
best_mod_pred = best_model.predict(X_test_scale)

centroids_scale = scaler.inverse_transform(centroids_std)

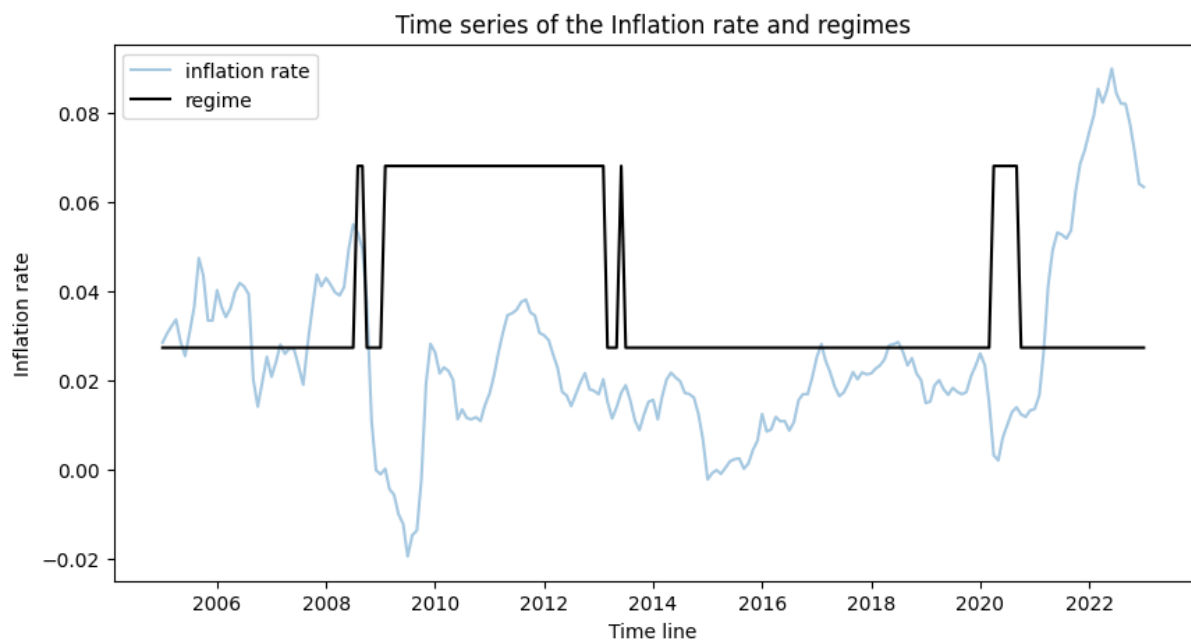
regimes = pd.DataFrame(test.iloc[:,0]).assign(regime = centroids_scale

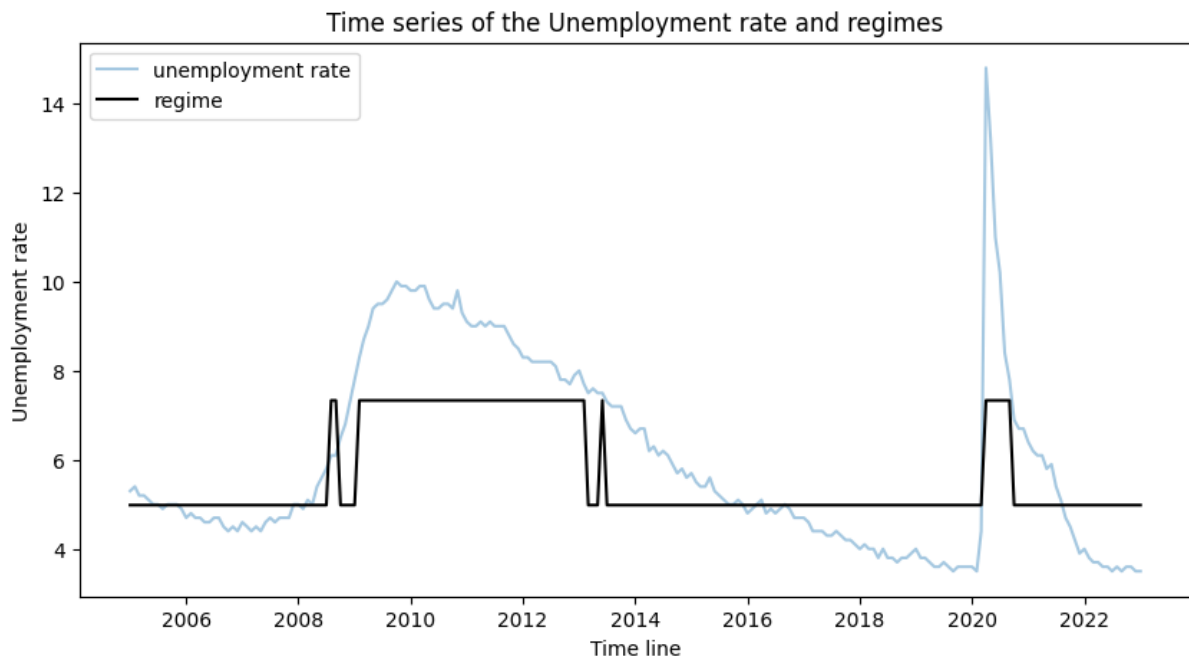
plt.figure(figsize=(10,5))
plt.plot(regimes["CPIAUCSL"], label='inflation rate', alpha=0.4)
plt.plot(regimes["regime"], '-k', label='regime')
plt.xlabel("Time line")
plt.ylabel("Inflation rate")
plt.title("Time series of the Inflation rate and regimes")
plt.legend()
plt.show()

regimes = pd.DataFrame(test.iloc[:,1]).assign(regime = centroids_scale

plt.figure(figsize=(10,5))
plt.plot(regimes["UNRATE"], label='unemployment rate', alpha=0.4)
plt.plot(regimes["regime"], '-k', label='regime')
plt.xlabel("Time line")
plt.ylabel("Unemployment rate")
plt.title("Time series of the Unemployment rate and regimes")
plt.legend()
plt.show()

```





(c) [3 marks]

A regime-switching model is a probabilistic model that governs how regime transitions take place. A very simple regime-switching model is a Markov chain. Here, we assume that the next month's regime depends on the current regime. In other words, all we need to know are the **transition probabilities** $p_{i,j} = \mathbb{P}(X_{t+1} = j | X_t = i)$, where X_t is the regime at day t . Using the time series of regimes in the training set that you found in (b), estimate these transition probabilities, as follows:

$$p_{i,j} = \frac{\mathbb{P}(X_{t+1} = j \text{ and } X_t = i)}{\mathbb{P}(X_t = i)} \approx \frac{\text{number of transition from regime } i \text{ to } j}{\text{number of transition from regime } i}$$

Next, let us check how good your regime switching model is. Calculate the transition probabilities using the test set. Do not retrain your k-means clusterer, simply use it to predict the regimes of the test set. Then, repeat the estimation of the transition probabilities on the test set.

Note: Depending on your choice of k , it is possible that some $p_{i,j}$ cannot be estimated due to having no transitions from regime i .

[Add your solution here]

```
In [ ]: from IPython.display import Markdown, display

# Function to estimate transition counts and probabilities
def estimate_transitions(labels, k):
```

```

counts = np.zeros((k, k), dtype=int)
for t in range(len(labels) - 1):
    counts[labels[t], labels[t+1]] += 1
row_sums = counts.sum(axis=1)

# Compute transition probabilities, handling zero-row cases
probs = np.zeros((k, k), dtype=float)
for i in range(k):
    if row_sums[i] > 0:
        probs[i] = counts[i] / row_sums[i]
    else:
        probs[i] = np.nan
return counts, probs

# Estimate for training set
counts_train, P_train = estimate_transitions(labels_train, best_k)
df_P_train = pd.DataFrame(P_train, index=[f"from {i}" for i in range(b
df_counts_train = pd.DataFrame(counts_train, index=[f"from {i}" for i

# Estimate for test set
counts_test, P_test = estimate_transitions(best_mod_pred, best_k)
df_P_test = pd.DataFrame(P_test, index=[f"from {i}" for i in range(bes

df_counts_test = pd.DataFrame(counts_test, index=[f"from {i}" for i in

# Display the transition probability matrices
display(Markdown("#### Transition probabilities for training set:"))
display(df_P_train)
display(Markdown("#### Count of transition for training set:"))
display(df_counts_train)
print("The total count of transition from 0 (training set): ", df_count
print("The total count of transition from 1 (training set): ", df_count

display(Markdown("#### Transition probabilities for testing set:"))
display(df_P_test)
display(Markdown("#### Count of transition for testing set:"))
display(df_counts_test)
print("The total count of transition from 0 (testing set): ", df_count
print("The total count of transition from 1 (testing set): ", df_count

```

Transition probabilities for training set:

	to 0	to 1
from 0	0.991903	0.008097
from 1	0.021053	0.978947

Count of transition for training set:

	to 0	to 1
from 0	490	4
from 1	4	186

The total count of transition from 0 (training set): 494

The total count of transition from 1 (training set): 190

Transition probabilities for testing set:

	to 0	to 1
from 0	0.974684	0.025316
from 1	0.068966	0.931034

Count of transition for testing set:

	to 0	to 1
from 0	154	4
from 1	4	54

The total count of transition from 0 (testing set): 158

The total count of transition from 1 (testing set): 58

5. Optional exercise: Neural Networks

There is nothing to submit for this exercise. However, it is strongly recommended that you spend time going through this exercise.

Visit the TensorFlow Playground at <https://playground.tensorflow.org/>

(a)

Layers and patterns: Try training the default neural network by clicking the run button (top left). Notice how it quickly finds a good solution for the classification task. Notice that the neurons in the first hidden layer have learned simple patterns, while the neurons in the second hidden layer have learned to combine the simple patterns of the first hidden layer into more complex patterns. In general, the more layers, the more complex the patterns can be.

(b)

Activation function: Try replacing the Tanh activation function with the ReLU

activation function, and train the network again. Notice that it finds a solution even faster, but this time the boundaries are linear. This is due to the shape of the ReLU function.

Try, also, the sigmoid (i.e. the logistic) activation function. Note that it will take significantly more time for the ANN to properly classify the dataset (you should wait until around 900 epochs).

(c)

Local minima: Modify the network architecture to have just one hidden layer with three neurons and the ReLU activation function. Train it multiple times. Between each training, press the reset button next to the play button on the top left. Notice that the training time and the shape of the decision borders varies a lot. If you are patient enough, you will notice that sometimes the network gets stuck in a local minimum which does not properly separate the classes (this happens at random).

(d)

Not enough neurons: Next, remove one neuron so that you have only one hidden layer with 2 neurons. Notice that the neural network is now incapable of finding a good solution, even if you try multiple times. The model has too few parameters and it systematically underfits the training set.

(e)

Enough neurons: Next, set the number of neurons to 8 and train the network several times. Notice that it is now consistently fast and never gets stuck. This highlights an important finding in neural network theory: large neural networks almost never get stuck at local optima, and even when they do these local optima are almost as good as the global optimum. However, they can still get stuck on long plateaus for a long time.

(f)

DNNs and vanishing gradients: Now change the dataset to be the spiral (bottom right dataset under "DATA"). Change the network architecture to have 4 hidden layers with 8 neurons each. Notice that training takes much longer, and

often gets stuck on plateaus for long periods of time. Also notice that the neurons in the highest layers (i.e. on the right) tend to evolve faster than the neurons in the lowest layers (i.e. on the left). This problem, called the “vanishing gradients” problem, can be alleviated using better weight initialization and other techniques, better optimizers (such as AdaGrad or Adam), or using Batch Normalization.